《人工智能》Project 2：强化学习

- 课程号：CS50020；时间：2026.11

项目介绍

本课程项目分为课上实验和课下作业两部分：课上实验部分将实现值迭代和 Q-Learning 相关的核心代码；课下作业包括了基于核心代码的扩展实验和需要手写完成的作业题。本次课程项目会首先在网格世界（GridWorld）中测试你的智能体，然后将其应用于一个勇者斗恶龙游戏（Hero vs Dragons）中。

重要时间点

内容	开始时间	截止时间	提交方式	提交内容
课前热身	11.9	11.12 12:00	Elearning 提交	热身题答案
课上实验	11.12 13:30	11.12 16:10	每组当堂验收	讲解算法及结果
课后作业	11.12	11.22 23:59	Elearning 提交	报告 + 代码

作业要求

1. 1-2 人组队完成；
2. 发布和提交平台：项目的发布和提交都通过 elearning 平台完成；
3. 课后作业提交要求：
   - 每组提交一份 **PDF 报告**，报告须包含以下内容：
     1. **课上实验回顾：**回顾课上实现的核心算法和代码，说明实现逻辑，并在报告中展示相应的关键代码；即使课上已验收，也必须在报告中回顾。
     2. **课下扩展实验：**说明完成情况和实现逻辑，并在报告中展示相应的关键代码。
     3. **自动评分结果：**附上能清楚显示最终各题得分和总分的自动评分结果截图；不能仅用文字自行陈述得分。
     4. **手写作业题：**给出详细过程，可以手写后扫描或拍照，也可以电子版输入。
     5. **组员信息与分工：**写明具体人员信息和各自分工。

     报告命名格式为任意一名同学的 `学号-姓名-PJ2 报告.pdf`；
   - 每组提交一份**代码附件压缩包**，只需将完成后的 `Agents/Coursework` 文件夹打包并上传，保留文件夹内的全部代码文件，无需提交其他实验文件。压缩包命名格式为与报告命名相同的同学的 `学号-姓名-PJ2 代码.zip`。**注意，请不要上传所有文件**。
4. 课程项目均需要使用 Python 编程，请提前安装好 Python 环境（推荐使用 Anaconda 或 uv）和集成开发环境（VS Code、PyCharm、Spyder 等）；

5. 课程项目的代码实现与手写作业题均**严禁抄袭**，相似度过高的作业提交将记为无效成绩；
6. 评分设置：课上核心代码实现当堂验收（30%）+ 课下作业报告提交（70%）。课上实验逾期未能验收时，仅验收最终报告内的课上实验介绍，并最多获得 $30\% \times 60\% = 18\%$ 的课上分数。

运行环境

请在 `PJ2-code-remake` 目录下执行本文命令。安装 Python 3 环境中的依赖：

```
python -m pip install -r requirements.txt
```

安装 PyTorch（可选）

我们不强制要求安装 PyTorch。Q7 的经验回放和探索率调度类可以在不安装 PyTorch 的环境中导入和实现；如果希望运行 Q7.3 的神经网络训练或加载模型测试，则需要安装 PyTorch。

安装时请先激活用于本项目的 Python 环境，再根据硬件选择以下一种方式。安装和运行训练时应使用同一个 Python 环境。

CPU 版本：不需要 NVIDIA 显卡，可在普通 CPU 上训练。

```
python -m pip install torch --index-url https://download.pytorch.org/whl/cpu
```

NVIDIA GPU 版本：需要支持 CUDA 的 NVIDIA 显卡以及兼容的显卡驱动。先运行 `nvidia-smi` 查看显卡和驱动信息，然后访问 PyTorch 官方安装页面，选择操作系统、Pip、Python 和适合机器的 CUDA 版本，执行页面生成的命令。例如，选择 CUDA 12.8 时可使用：

```
python -m pip install torch --index-url https://download.pytorch.org/whl/cu128
```

实验介绍

在这个项目中，需要实现值迭代和 Q-Learning 相关的核心代码。

与之前的项目一样，本项目包含一个自动评分工具，可以用它来进行评分。运行以下命令，为所有问题评分：

```
python autograder.py
```

通过以下命令只运行特定问题的评分，例如 q2：

```
python autograder.py -q q2
```

参数 `-q` 控制待评估的题目。

当前自动评分覆盖 Q1–Q7，满分为 **25 分**；手写 Q8 另行评阅，不计入这 25 分。

在实现过程中，只需要关注其中的几个文件，可以忽视其他的支持文件：

需要编辑的文件

文件	说明
`Agents/Coursework/value_iteration_agents.py`	价值迭代的智能体，用来解决 MDPs。
`Agents/Coursework/qlearning_agents.py`	Q-learning 的智能体。
`Agents/Coursework/analysis.py`	用来提交 Q4 的答案。
`Agents/Coursework/feature_extractors.py`	Q6 特征提取器设计的入口。
`Agents/Coursework/dqn_utils.py`	Q7 的经验回放与探索率调度实现。

可以参考的文件

文件	说明
`Game/Core/mdp.py`	定义了 MDPs 的有关函数。
`Agents/Runtime/learning_agents.py`	定义了 `ValueEstimationAgent` 和 `ReinforcementAgent` 等基础类。
`Game/Core/utilities.py`	定义了一些有用的类。
`Agents/Runtime/feature_extractors.py`	用于抽取特征的函数。
`Agents/Runtime/dqn_agent.py`	Q7 的神经网络训练和测试流程。

Part 1. 课前热身

Q0. 上手体验格子世界

下载并解压 zip 文件，进入文件夹目录后，在终端命令行输入以下命令来体验格子世界，通过方向键进行控制：

```
python gridworld.py -m
```

你可以看到两个出口，分别为奖励 1 和 -1。蓝点表示智能体，当你按向上键的时候，智能体实际向北移动的概率为 80%。

你可以控制模拟的多个参数。运行以下命令可以查看完整的选项列表：

```
python gridworld.py -h
```

以下命令可以让默认的智能体随机移动：

```
python gridworld.py -g MazeGrid
```

你可以看到随机智能体在网格中四处碰撞，直到偶然找到出口，这并不是智能体最理想的表现。

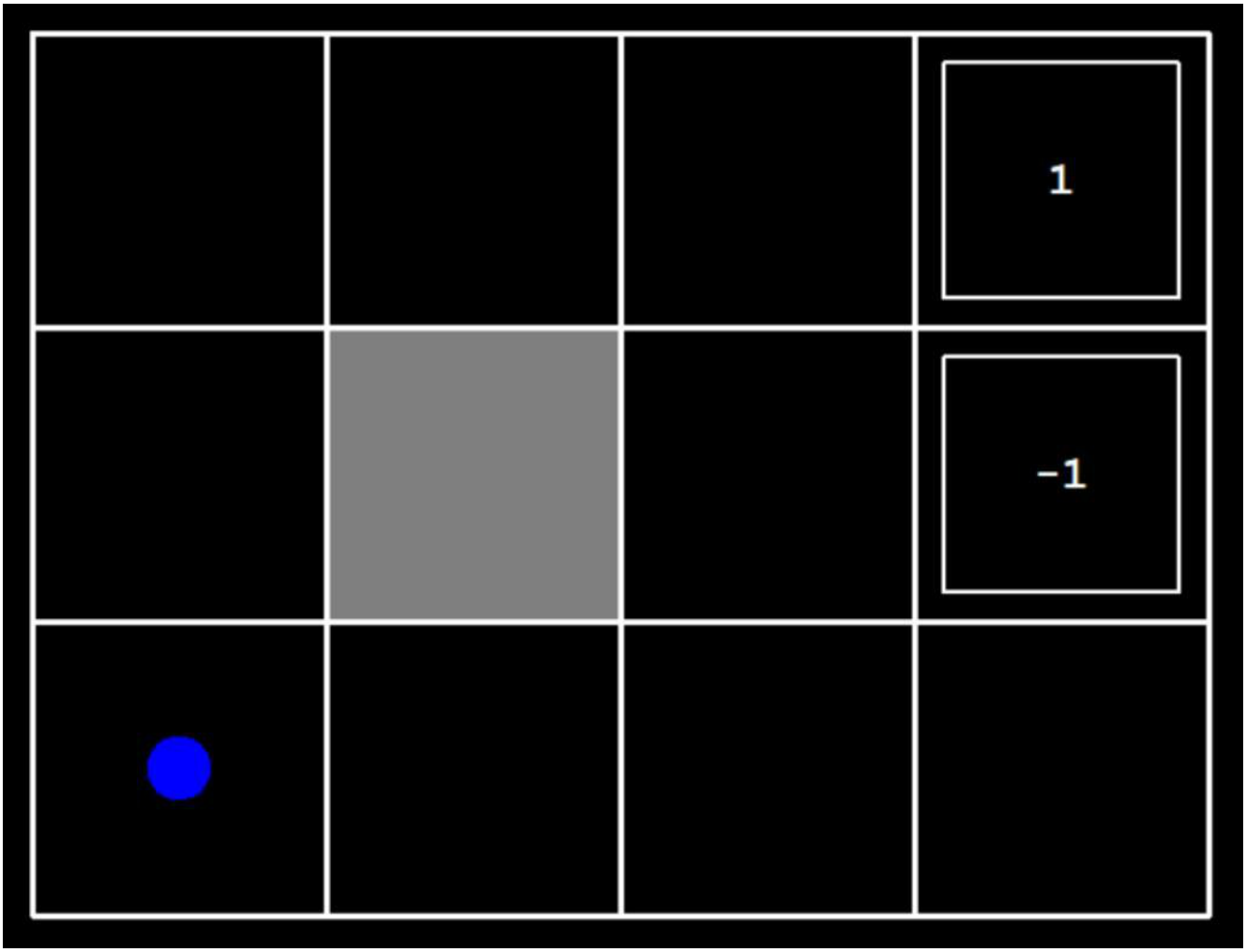


图 1: 格子世界初始界面

注意: 在网格世界的 MDP 中，智能体首先必须进入一个预终止状态（在 GUI 中以双框显示），然后执行特殊的退出动作，才会使得整个回合真正结束（进入称为 `TERMINAL_STATE` 的终端状态，在 GUI 中未显示）。如果你手动运行一个回合，由于折扣率的影响（使用 `-d` 参数更改，默认为 0.9），总回报可能低于预期。

查看控制台输出（或使用 `-t` 显示纯文本输出），你可以看到智能体每次状态转换的详细信息（可以使用 `-q` 关闭此信息）。

与勇者斗恶龙游戏一样，位置由 `(x, y)` 笛卡尔坐标表示，数组的索引为 `[x][y]`，其中 `'north'` 表示 y 增加的方向。默认情况下，大多数状态转换的奖励为零，但你可以使用 `-r` 来更改这一设定。

课前热身提交内容

请阅读相关代码，并在 elearning 上提交以下问题的答案（一行代码或多行代码）。

以下问题均针对 Gridworld：给定 MDP 对象 `mdp` 和状态 `state`，请通过 MDP 提供的接口作答。

问题 0.1: 给定一个状态 `state`，如何获取所有可能的动作（`action`）?

问题 0.2: 如何判断一个状态 `state` 是否是终止状态?

问题 0.3: 给定一个状态-动作对 `(state, action)`，怎么得到可能的后续状态及对应的概率？请以二元组的形式保存在一个列表中。

Part 2. 课上实验内容

Q1（5 分）: 值迭代算法

需要完成: `Agents/Coursework/value_iteration_agents.py` 中的 `runValueIteration`、`computeQValue`、`computeActionFromValues`。

回忆一下值迭代中的状态更新公式:

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s')$$

$$V^*(s) = \max_a Q^*(s,a)$$

完成在 `Agents/Coursework/value_iteration_agents.py` 中定义的值迭代智能体 `ValueIterationAgent`。`ValueIterationAgent` 在实例化时接受一个 MDP 作为输入，并在执行指定次数的值迭代后返回。

除了值迭代更新的 `runValueIteration`，还需实现以下方法:

- `computeActionFromValues(state)`: 根据 `self.values` 中的值函数计算给定状态下的最优动作。
- `computeQValue(state, action)`: 根据 `self.values` 中的值函数返回状态-动作对 `(state, action)` 的 Q 值。

重要说明：请使用同步更新的值迭代方法，即每个向量是从固定的前一轮向量计算得到的。

提示：可以选择使用 `Game/Core/utilities.py` 文件中的 `util.Counter` 类，这是一个默认值为零的字典。使用 `argMax` 时要小心，因为实际需要的 argmax 值可能是 `Counter` 中不存在的键值。

提示：可能存在 MDP 中某个状态没有可用动作的情况。

运行 `autograder` 来测试你的实现是否正确：

```
python autograder.py -q q1
```

以下命令会加载你实现的 `ValueIterationAgent`，进行 100 次值迭代，并运行 10 个回合。按任意键可以在显示值（values）、Q 值（Q-values）和策略之间切换，V 值是显示在方格内的数字，Q 值显示在方格的四分之一部分中，策略则用箭头从每个方格中指出。理论上，可以看到起始状态的值 `V(start)` 与运行这些回合后打印的实际平均回报相当接近。

```
python gridworld.py -a value -i 100 -k 10
```

提示：在默认的 `BookGrid` 上运行 5 次值迭代后，你应该会得到以下输出：

```
python gridworld.py -a value -i 5
```

评分：你的值迭代智能体将在一个新的网格上进行评分。我们会在固定的迭代次数后以及收敛时（例如 100 次迭代后）检查你的值函数、Q 值和策略。

Q2（5 分）：Q-Learning

需要完成： `Agents/Coursework/qlearning_agents.py` 中的 `getQValue`、`computeActionAndValueFromQValues`、`update`。

回忆一下 Q-learning 中的更新公式：

$$V(s) = \max_a Q(s, a)$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right)$$

接下来，你需要编写一个 Q-learning 智能体。与值迭代智能体不同，这个智能体在构建时不需要太多计算，而是通过与环境的交互，通过调用其 `update(state, action, nextState, reward)` 方法，以试错的方式学习。`QLearningAgent` 类的 Q-Learning 智能体框架已经在 `Agents/Coursework/qlearning_agents.py` 中给出。在本题中，你需要实现 `update`、`computeActionAndValueFromQValues`、`getQValue` 方法。

提示：在实现 `computeActionAndValueFromQValues` 时，为了获得更好的策略，你应该在遇到相等情况时进行随机选择。可以使用 `random.choice()` 函数来实现。

提示：请确保在 `computeActionAndValueFromQValues` 函数中，你仅通过调用 `getQValue` 来访问 Q 值，这会在评分的时候用到。

你可以使用键盘手动控制来观察 Q-Learning 智能体的学习过程：

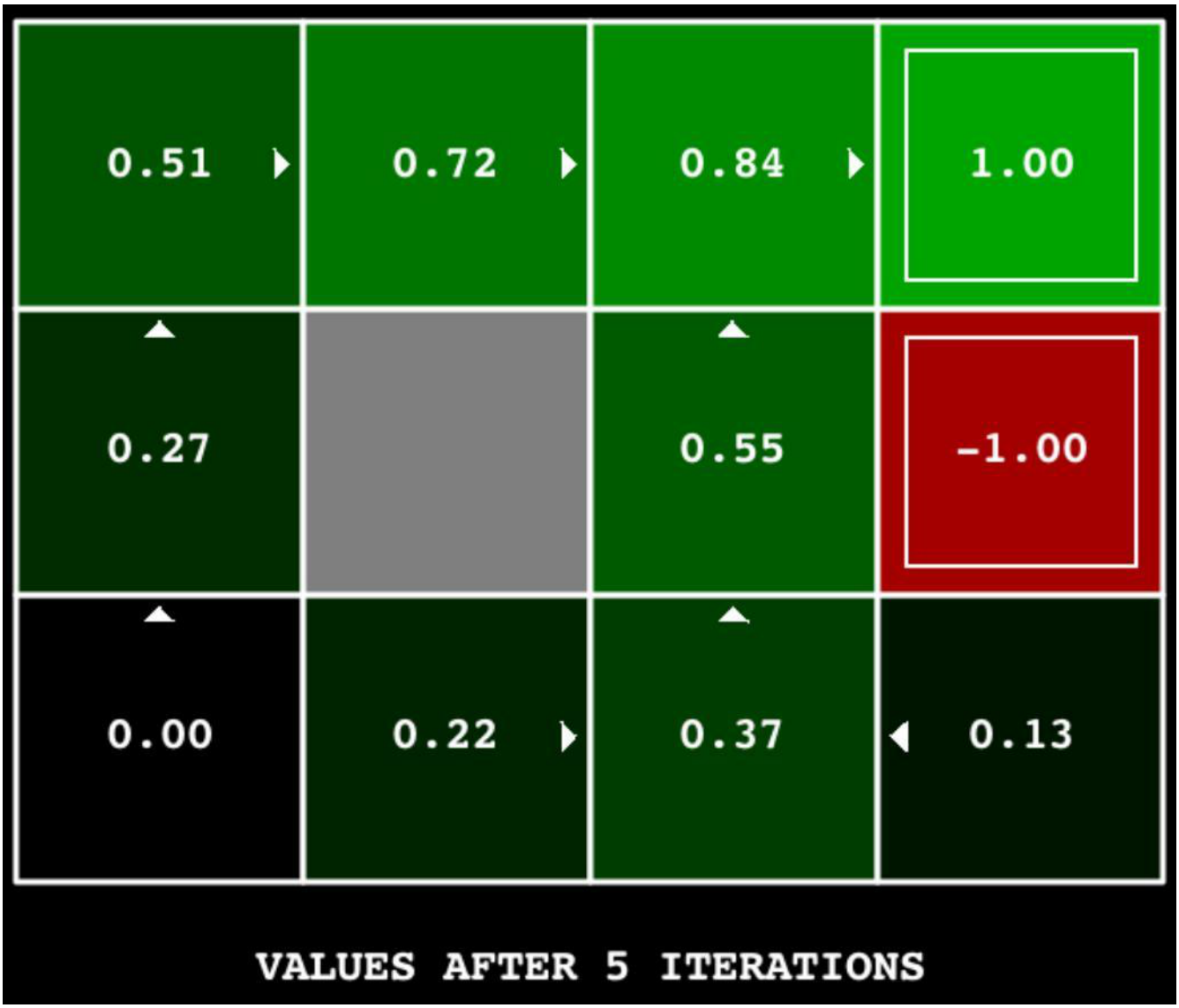


图 2: BookGrid 运行 5 次值迭代后的输出

```
python gridworld.py -a q -k 5 -m --noise 0.0
```

`-k` 参数用来控制你的智能体可以学习的回合数。为了方便调试，你可以通过使用 `--noise 0.0` 参数来关闭噪声（智能体没有进行预期行为，随机做出其他行为的概率）。如果你在四个回合内手动引导网格中的智能体沿着最优路径先向北再向东，你应该会看到如下的 Q 值：

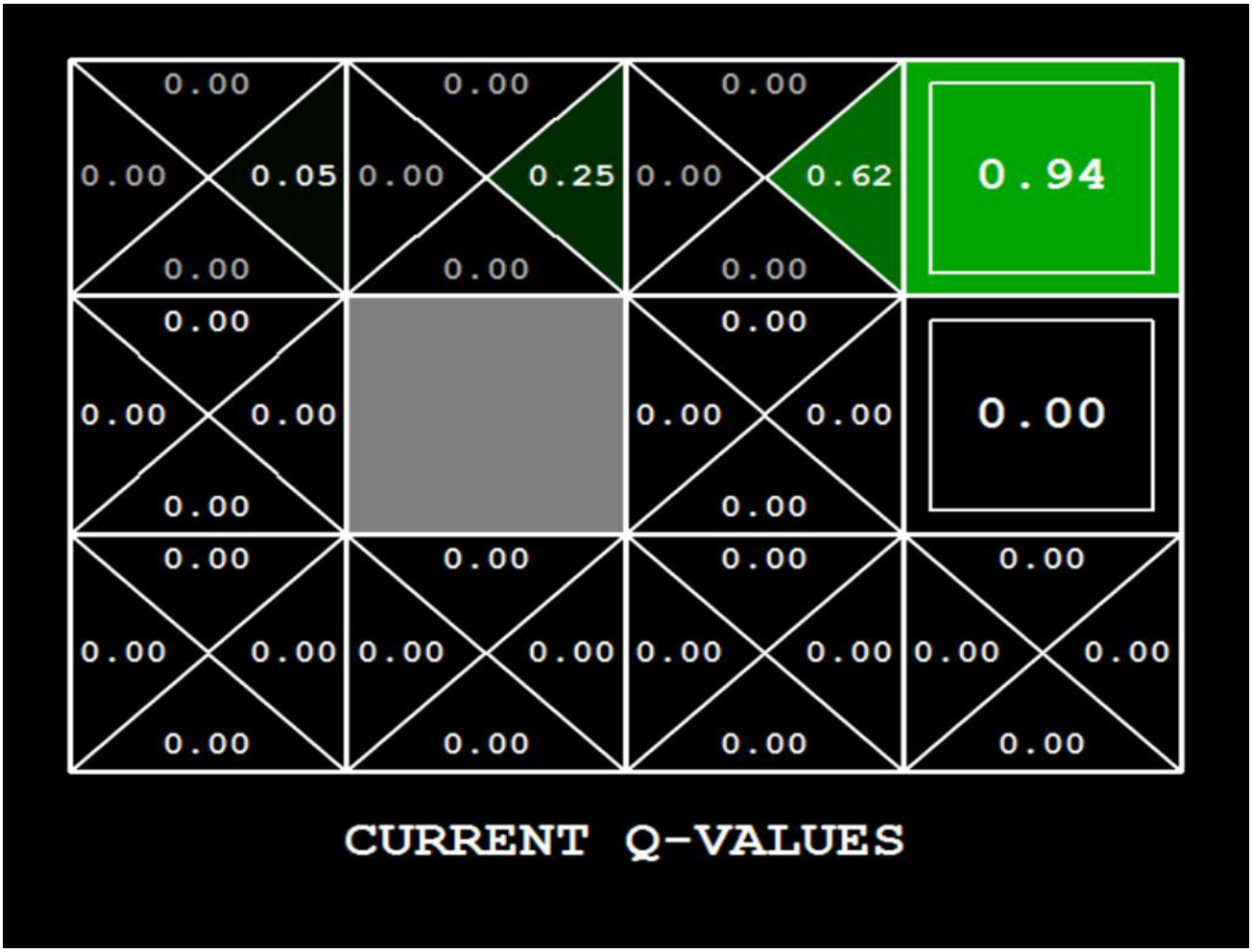


图 3: 手动引导四个回合后的 Q 值

评分：我们会运行你的 Q-learning 智能体，并检查它在面对相同示例集时是否能学习到与参考代码相同的 Q 值和策略。运行 `autograder` 来检查你的实现：

```
python autograder.py -q q2
```

Part 3. 课下作业内容

Q3（2 分）：ϵ-贪婪策略

需要完成： `Agents/Coursework/qlearning_agents.py` 中的 `getAction`。

在 Q-learning 智能体的 `getAction` 函数中实现 ϵ-贪婪（epsilon-greedy）动作选择策略。智能体应该以一定（ϵ）的概率选择随机动作，在其它时候选择当前最好的 Q 值对应的动作。请注意，选择随机动作时同样可能选择到最好的动作，也就是说，不应该故意选择一个次优动作，而是从所有合法动作中随机选择。

你可以使用 `random.choice` 函数在列表中随机地选择一个元素。`util.flipCoin(p)` 函数可以以概率 p 返回 `True`，以概率 1-p 返回 `False`。

实现 `getAction` 方法后，在格子世界中观察智能体的行为（$\epsilon = 0.3$）：

```
python gridworld.py -a q -k 100
```

你还可以使用 `-e` 参数调整 ϵ 值，例如以下命令：

```
python gridworld.py -a q -k 100 --noise 0.0 -e 0.1
python gridworld.py -a q -k 100 --noise 0.0 -e 0.9
```

运行 `autograder` 检查你的实现：

```
python autograder.py -q q3
```

Q4（5 分）：最优策略

需要完成：补全 `Agents/Coursework/analysis.py` 中 `question2a()` 到 `question2e()` 的参数组。

以下图 `DiscountGrid` 为例，该网格有两个具有正回报的终止状态，一个靠近的出口，回报为 +1，另一个较远的出口，回报为 +10。网格的底部由具有负回报的终止状态组成，这个悬崖区域的每个状态的回报为 -10。起始状态是黄色方格。

我们区分以下两种路径：

1. **冒险路径：**沿网格底部移动。这些路径更短，但有可能得到较大的负回报，如图中的红色箭头所示。
2. **安全路径：**沿网格顶部移动。这些路径较长，但更容易避开底部的负回报，如图中的绿色箭头所示。

选择何种路径会受到折扣因子的影响，较高的折扣因子可能会更策略倾向于远离悬崖以获取更高的远期收益，而较低的折扣因子可能会更注重短期的奖励和风险。

在这个问题中，你需要为该 MDP 问题选择折扣因子、噪声（智能体没有进行预期行为，随机做出其他行为的概率）和生存奖励三个参数，以产生不同类型的最优策略。

在 `Agents/Coursework/analysis.py` 中，`question2a()` 到 `question2e()` 分别返回一个包含（折扣因子，噪声，生存奖励）的三元组。每个子问题的参数设置下，应该使得智能体在遵循其最优策略并且不受任何噪声的影响时，表现出指定的行为。如果某些行为无法通过参数设置实现，需返回字符串 `"NOT POSSIBLE"` 以表明不可行。

以下是你需要尝试产生的最优策略类型：

1. 偏好接近的出口（+1），走冒险路径。
2. 偏好接近的出口（+1），避开悬崖。
3. 偏好远处的出口（+10），走冒险路径。
4. 偏好远处的出口（+10），避开悬崖。

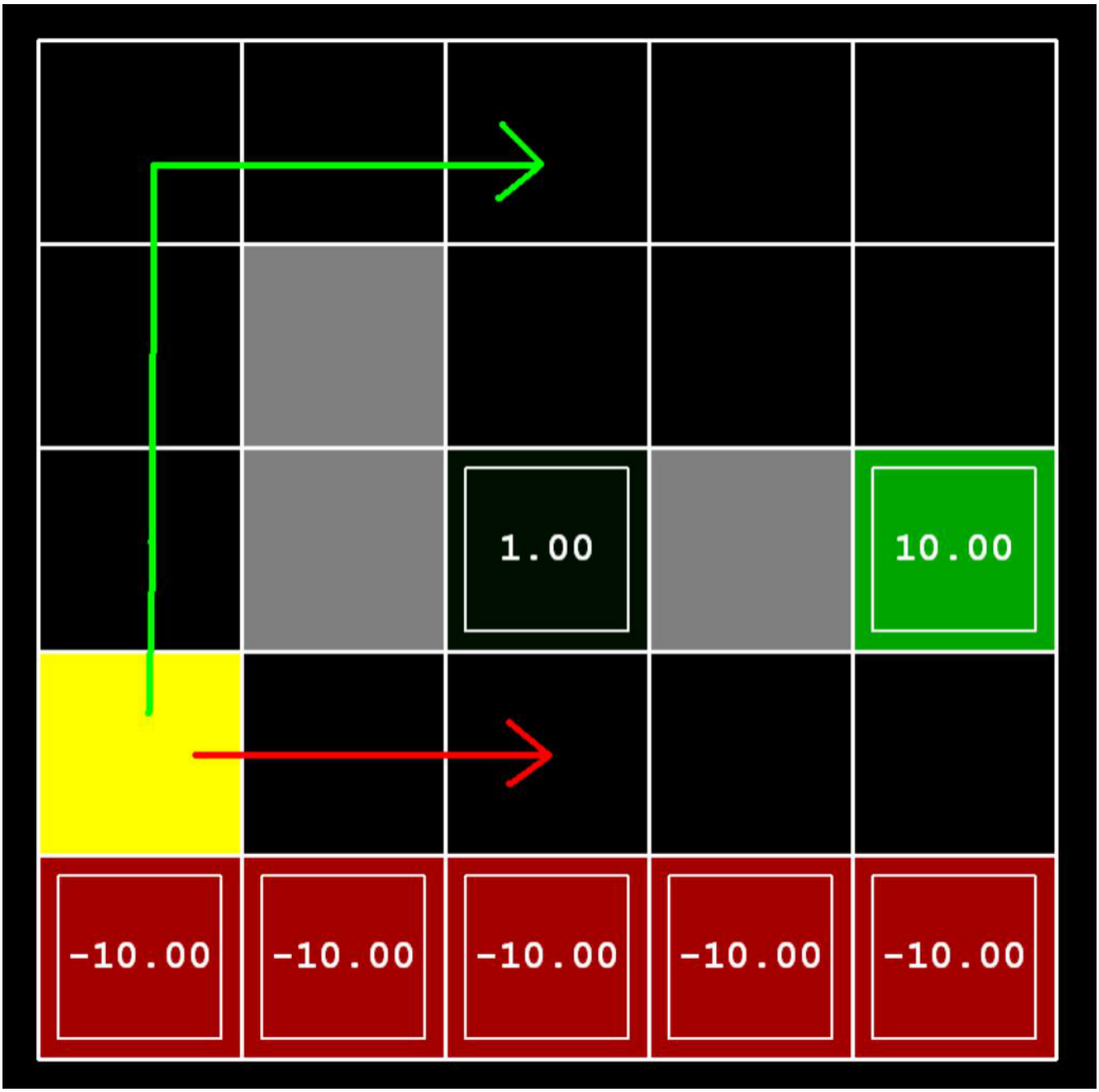


图 4: DiscountGrid 中的冒险路径与安全路径

5. 避开两个出口和悬崖（因此一个回合永远不会结束）。

要查看一组参数最终会表现出什么样的行为，请运行以下命令：

```
python gridworld.py -g DiscountGrid -a value --discount [YOUR_DISCOUNT] --noise [YOUR_NOISE] --
    livingReward [YOUR_LIVING_REWARD]
```

要测试你的参数设置，运行 `autograder`：

```
python autograder.py -q q4
```

注意：你可以在 GUI 中检查你的策略。在某些电脑上你可能看不到箭头。在这种情况下，按下键盘上的任意键切换到 Q 值显示，并通过在每个状态下取可用 Q 值的最大值来确定策略。

评分：我们将检查在每种情况下是否返回了所需的策略。

Q5（1 分）：Q-Learning 与勇者斗恶龙

需要完成：检查 `Agents/Coursework/qlearning_agents.py` 中的 `getAction` 和 `computeActionAndValueFromQValues`。

勇者斗恶龙游戏分为两个阶段。第一阶段是训练，在此阶段勇者学习各个位置和动作的价值。即便是极小的网格也需要很长的时间才能学习到准确的 Q 值，所以默认情况下，勇者的训练游戏在静默模式下进行，不会显示 GUI。

当训练完成后，勇者会进入测试模式，在测试中，`self.epsilon` 和 `self.alpha` 将被设置为 0.0，从而停止 Q-learning 并禁用探索模式，让勇者只利用已经学到的策略。默认情况下，测试部分将显示在 GUI 中。完成 Q2 和 Q3 后，无需为 Q5 新增算法代码，即可按以下命令运行 Q-learning 的勇者来处理非常小的网格：

```
python run.py -p HeroQAgent -x 2000 -n 2010 -l smallGrid
```

`HeroQAgent` 已根据你编写的 `QLearningAgent` 进行了定义，`HeroQAgent` 唯一不同的是它有更适用于勇者斗恶龙问题的默认学习参数（`epsilon=0.05`，`alpha=0.2`，`gamma=0.8`）。如果上述命令正常运行，且你的智能体在 100 局测试中至少获胜 70 局（胜率至少 70%），你将获得本题的全部分数。评分器会在 2000 次训练后运行 100 次进行测试。

提示：如果你的 `QLearningAgent` 在 `gridworld.py` 中运行良好，但在 `smallGrid` 上似乎没有学习到一个良好的策略，可能是因为你的 `getAction` 或 `computeActionAndValueFromQValues` 在某些情况下没有适当考虑未见过的动作。

比如，未见过的动作的初始 Q 值为零，而已见过的所有动作都具有负 Q 值，那么未见过的动作可能是最优选择。

运行 `autograder` 进行检查：

```
python autograder.py -q q5
```

注意：如果你想调整学习参数，可以使用 `-a` 进行调整，例如 `-a epsilon=0.1,alpha=0.3,gamma=0.7`。它们分别对应智能体内部的 `self.epsilon`、`self.alpha` 和 `self.discount`。

注意：尽管游戏总数为 2010 局，但由于 `-x 2000` 参数，前 2000 局游戏用于训练（不显示逐局画面和结果，但仍会输出训练统计）。因此，你将只能看到最后 10 局的游戏画面。训练游戏数将作为 `numTraining` 参数传递给智能体。如果你想观看 10 局训练以观察学习进展，可以使用以下命令：

```
python run.py -p HeroQAgent -n 10 -l smallGrid -a numTraining=10
```

在训练过程中，每 100 局会输出统计信息，显示勇者的表现情况。在训练时，由于 epsilon 是正值，勇者即使已经学到了较好的策略，偶尔也会进行随机探索，可能会因此碰上恶龙。作为参考，通常需要 1000 到 1400 局左右，勇者每 100 局的奖励才会变为正数。到训练结束时，这个值应该持续为正数，并且相对较高（在 100 到 350 之间）。

Q6（6 分）：值近似与特征设计

本题分为 **基础算法 3 分**+**特征设计 3 分**。

Q6.1（3 分）：基础算法

需要完成：完成 `Agents/Coursework/qlearning_agents.py` 中的 `ApproximateQAgent`，编写 `getQValue`、`update`。

实现一个值近似的 Q-learning 智能体，该智能体可以给状态的特征学习权重。其中多个状态可能共享相同的特征。请在 `Agents/Coursework/qlearning_agents.py` 文件中的 `ApproximateQAgent` 类中编写实现，该类是 `HeroQAgent` 的子类。

值近似的 Q-learning 假设存在一个状态和动作对的特征函数 $f(s,a)$，它会返回一个向量 $[f_1(s,a),\dots,f_i(s,a),\dots,f_n(s,a)]$，即特征值的向量。特征向量是 `util.Counter`（类似字典）对象，包含非零的特征-值对；所有未列出的特征默认值为零。

值近似的 Q 函数形式如下：

$$Q(s,a)=\sum_{i=1}^{n} w_i f_i(s,a)$$

其中，每个权重 w_i 与特定特征 $f_i(s,a)$ 相关联。在代码中，你应该将权重向量实现为一个**字典**，将特征（特征提取器将返回的键）映射到权重值。权重的更新方式与 Q 值更新类似：

$$w_i \leftarrow w_i + \alpha \cdot \mathit{difference} \cdot f_i(s,a)$$

$$\mathit{difference} = \Big(r + \gamma \max_{a'} Q(s',a')\Big) - Q(s,a)$$

默认情况下，`ApproximateQAgent` 使用 `IdentityExtractor`，该提取器为每个 `(state, action)` 对分配一个单一特征。在相同学习参数下，使用此特征提取器时，你的值近似的 Q-learning 智能体应该与 `HeroQAgent` 的行为相同。你可以通过以下命令测试：

```
python run.py -p ApproximateQAgent -x 2000 -n 2010 -l smallGrid
```

提示： `ApproximateQAgent` 是 `QLearningAgent` 的子类，因此它共享多个方法，例如 `getAction`。确保 `QLearningAgent` 中的方法通过调用 `getQValue` 来获取 Q 值，而不是直接访问 Q 值。

基础评分： 我们将运行值近似的 Q-learning 智能体，并检查它在遇到相同示例集时学习到的 Q 值和特征权重是否与我们的参考实现一致。全部通过获得基础 3 分；否则基础分为 0，且不运行 Q6.2 的评估。

Q6.2（3 分）：特征选择与设计

需要完成： 修改 `Agents/Coursework/feature_extractors.py` 中的 `StudentFeatureExtractor`。该类默认使用只有常数偏置的 `BiasExtractor`，作为可运行的弱基线。

在值近似方法中，特征的好坏会直接影响到近似效果。

在 Q6.1 基础算法正确的前提下，你需要设计合适的特征，以达到尽可能好的学习效果。

我们预先准备了一些可用的特征提取器。你可以选用这些预置的特征提取器，或是组合多个特征提取器，也可以在 `getFeatures(state, action)` 中实现自定义的特征提取器。

特征提取器的公共基类位于 `Agents/Runtime/feature_extractors.py`，接口约定如下：

- 接收勇者斗恶龙游戏的 GameState 和一个合法动作。
- 返回“可哈希特征键 → 有限实数”的稀疏映射，推荐使用 `util.Counter`。

注意： 特征提取器应能无参数构造；需要参数的组合器可在 `StudentFeatureExtractor` 内部组装。

你可以使用以下预置的特征提取器：

提取器	提取的特征	取值与说明	注
`BiasExtractor`	常数偏置 `bias`。	始终返回 `bias=0.1`。	作为弱基线。
`CoordinateExtractor`	勇者当前位置 `(x, y)`、横坐标 `x=…`、纵坐标 `y=…`、候选动作 `action=…`。	四个特征值均为 `1.0`。	坐标出现在特征键中，数值不是坐标本身。
`CoinExtractor`	常数偏置 `bias`、候选位置是否有金币 `collects-coin`、候选位置到最近可达金币的 BFS 距离 `closest-coin`。	偏置为 `0.1`；金币指示值为 `0` 或 `0.1`；距离为 `BFS距离 / 地图面积 / 10`。	无可达金币时不返回距离键 `closest-coin`。
`DragonExtractor`	常数偏置 `bias`、可在一步内到达候选位置的恶龙数量 `#-of-dragons-1-step-away`。	偏置为 `0.1`；危险特征为 `恶龙数量/ 10`。	`#-of-dragons-1-step-away` 按每只恶龙所在格及其无墙四邻格判断，包含候选位置已被恶龙占据的情况。

提取器	提取的特征	取值与说明	注
`IdentityExtractor`	为输入的 `(state, action)` 对建立一个独立特征。	以 `(state, action)` 为键，值为 `1.0`。	该提取器中，不同状态动作对不共享权重。作为表格型表示的对照。
`CombinedExtractor`	组合多个特征提取器。保留所选提取器的全部特征，将每个键改为 `(提取器序号, 原特征键)`。	接收特征提取器实例列表。各组件的值保持不变。	/

注意：上表中的“候选位置”指勇者执行给定合法动作后的位置；未返回的特征键按 `0` 处理。

提示：预置的特征提取器均定义在 `Agents/Runtime/feature_extractors.py` 中，**强烈建议**参阅它们的实现方式。

例如，可以在 `StudentFeatureExtractor` 的构造函数中选择 `self.extractor = CoordinateExtractor()` 来直接选用预置的特征提取器，也可以尝试用以下方法组合两个特征提取器：

```
self.extractor = CombinedExtractor([BiasExtractor(), CoordinateExtractor()])
```

自定义方案可在 `getFeatures` 中直接计算并返回 Counter。可以考虑动作方向、金币距离、恶龙风险、弱化状态或剑等信息。

提示：自定义特征提取器时，需要注意数值尺度，避免过大的特征值使更新不稳定。

注意：自定义的 `getFeatures(state, action)` 不得修改输入状态。对同一个游戏状态和动作，应返回相同的特征；如需随机参数，应在提取器初始化时固定。因为评分器会复用同一观测下的重复特征查询，并返回独立副本。

使用你设计的 `StudentFeatureExtractor` 运行值近似的 Q-learning 智能体，观察其训练与测试表现（具体胜率会受到随机种子和智能体实现细节的影响）：

```
python run.py -p ApproximateQAgent -a extractor=Agents.Coursework.feature_extractors.
    StudentFeatureExtractor -x 2000 -n 2100 -l mediumGrid
```

注意：在命令行中设置 `extractor=CoordinateExtractor` 只影响那次运行。若希望正式评分使用 `CoordinateExtractor`，请在 `StudentFeatureExtractor` 中选择它。

附加评分：我们按照平均通关率给阶梯分：

900 局中的胜局数	平均通关率	附加分
0–179	小于 20%	0
180–449	至少 20%，小于 50%	1
450–719	至少 50%，小于 80%	2
720–900	至少 80%	3

评分采用以下固定协议，学习参数不能通过提交配置改变：

项目	设置
地图及顺序	mediumGrid、smallClassic、mediumClassic
每张地图	3 次独立重复，每次重新创建 Agent 和零权重
每次重复	2000 局训练 + 100 局测试，总计 18000 局训练、900 局测试
学习参数	alpha=0.2、gamma=0.99、epsilon=0.05
对手	地图规定数量的 RandomDragon
每局上限	累计 900 次勇者行动
测试	epsilon、alpha 归零

附加实验在无图形独立进程内运行，九组训练最多同时运行 6 组（不超过电脑的逻辑 CPU 数量），正常目标耗时约 1–5 分钟。完整 Q6 评分的总时限为 **900 秒（15 分钟）**。运行异常或超时会使附加分为 0；已经通过 Q6.1 获得的基础 3 分保留。

运行以下指令测试 `StudentFeatureExtractor` 的得分：

```
python -m Game.Application.feature_evaluation
```

这条命令不会执行 Q6.1 基础测试和 Q2、Q3 前置检查。它会显示你能获得的 0–3 分附加分数，但不能代替 `autograder` 的完整 Q6 评分。

运行 `autograder` 进行完整 Q6 评分：

```
python autograder.py -q q6 --no-graphics
```

Q7：深度强化学习（1 分）

在 Q6 中，我们手工设计特征，例如到最近金币的距离、附近恶龙的数量，再用这些特征的线性组合近似 Q 值。本题从两个视角继续推进：

1. **从手工特征到更原始的状态信息**。不再由我们计算“到最近金币的距离”等高层特征，而是将地图信息直接组织成网格通道，交给神经网络学习。本项目使用勇者、金币、普通龙、墙、虚弱龙剩余时间和宝剑六个通道；这些通道仍是对游戏状态的一种编码，并非未经处理的原始画面。
2. **从线性函数到神经网络**。 Q6 的 $Q(s,a) = \sum_i w_i f_i(s,a)$ 对特征采用线性组合；本题用卷积神经网络 $Q_\theta(s,a)$ 近似 Q 值，让网络学习更复杂的状态与动作关系。

   “The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin.”

   ——Richard Sutton, The Bitter Lesson（2019）

需要完成： `Agents/Coursework/dqn_utils.py` 中的 `EpsilonScheduler.value` 和 `ReplayBuffer` 四个方法。保留给定的类名、方法名及参数接口，以便训练程序调用。`Agents/Runtime/dqn_agent.py` 中的网络和训练流程可供阅读，本题不要求修改。Q7.1 和 Q7.2 各占 0.5 分，独立计分；Q7.3 不计分。

完成后可运行以下命令检查 Q7，无需安装 PyTorch：

```
python autograder.py -q q7 --no-graphics
```

Q7.1 ϵ 衰减（0.5 分）

强化学习需要在探索与利用之间取得平衡：探索有助于发现更好的行动，利用则根据当前学到的知识选择行动。训练初期，智能体对环境了解较少，需要较高的探索率来收集多样化的经验；随着训练进行，智能体逐渐学到有价值的行为，可以降低探索率，更多地利用已有知识。因此，我们让 ϵ 随采样步数线性衰减，以满足我们的需求。

请完成 `EpsilonScheduler.value(step)`：设起点为 ϵ_{start}，终点为 ϵ_{end}，衰减步数为 D，可用以下关系计算：

$$\epsilon(t) = \epsilon_{\text{start}} + (\epsilon_{\text{end}} - \epsilon_{\text{start}}) \operatorname{clip}\left(\frac{t}{D}, 0, 1\right)$$

其中 t 是已经采集的环境步数，$\operatorname{clip}(x, 0, 1)$ 将 x 限制在 $[0, 1]$。第 0 步返回起点，在第 D 步及之后返回终点。`DQNAgent` 每采集一条转移更新一次探索率；设置固定 `--epsilon` 时不启用衰减。

Q7.2 经验回放（0.5 分）

智能体与环境交互时，连续得到的经验往往十分相似。例如，勇者沿着走廊连续移动时，相邻几步看到的地图和采取的动作变化不大。如果神经网络总是按时间顺序只用最新经验更新，连续多次更新就可能集中反映当前遇到的局部情形，使学习过程不稳定。此外，一条经验只用于一次更新，也没有充分利用采集它所花费的交互成本。

经验回放将过去的状态转移存入一个有限容量的经验池，训练时从中随机抽取一批。这样，一次更新可以结合不同时刻收集的经验，减弱连续样本的相关性；同一条经验也有机会在之后被再次使用。

请在 `Agents/Coursework/dqn_utils.py` 中完成 `ReplayBuffer`，实现以下接口：

方法	要求
`ReplayBuffer(capacity)`	初始化空经验池，容量上限为 `capacity`。
`add(transition)`	加入一条转移；已满时移除最早加入的转移。
`sample(batch_size)`	均匀、无放回地随机抽取 `batch_size` 条，返回转移列表；采样不删除经验。
`len(buffer)`	返回当前保存的转移数量，而不是容量。

`transition` 表示一条完整的经验；经验池只需原样保存和返回。可以使用 Python 标准库中的 `collections.deque` 或其他合适的数据结构。

Q7.3 DQN 训练与测试

本题为选做题，不计分

在 Q-learning 中，我们希望用即时奖励和下一状态的估计价值构造学习目标，再让当前状态的 Q 值逐渐接近这个目标。DQN 沿用这一思路，但用神经网络近似 Q 函数。若直接用同一网络同时计算当前 Q 值和下一状态的目标值，每次更新网络参数都可能立即改变目标，导致学习目标不断移动，训练容易不稳定。

为减缓这个问题，本项目维护两个结构相同、参数各自保存的网络：

- **在线网络（online network）** Q_θ：根据当前状态估计各动作的 Q 值，用于选择动作；训练时通过梯度下降更新参数 θ。
- **目标网络（target network）** $Q_{\bar{\theta}}$：用于计算下一状态的目标 Q 值。它在一段时间内保持参数不变，定期将在线网络的参数复制过来，使学习目标变化得更平缓。目标网络的参数不会通过反向传播直接更新。

本项目采用 **Double DQN**。对于一条经验 (s, a, r, s')，先由在线网络在下一状态 s' 的合法动作中选择价值最高的动作，再由目标网络评估该动作：

$$a^* = \arg\max_{a' \in A(s')} Q_\theta(s', a'), \qquad y = r + \gamma Q_{\bar{\theta}}(s', a^*)$$

其中 $A(s')$ 是下一状态的合法动作集合。若下一状态没有合法动作，则没有后续价值，目标为 $y = r$。将动作选择和价值评估分开，旨在缓解直接对同一组估计值取最大值时可能产生的高估；这并不保证完全消除高估。

训练流程可以概括为：

1. 根据当前探索率，随机选择合法动作，或使用在线网络选择 Q 值最高的合法动作；执行动作后，将得到的状态转移加入经验池。
2. 经验池达到预热数量后，按训练间隔从中随机抽取一批经验。对每条经验计算上述目标值，再将在线网络对已执行动作的预测与目标值比较。
3. 使用这批样本的均方误差更新在线网络。目标值计算不参与梯度传播，因此本次更新只改变在线网络。
4. 每经过一定次数的**网络参数更新**，将在线网络的参数复制到目标网络，然后继续收集经验和训练。

经验回放让每批数据可以来自不同时刻，减弱相邻经验的相关性；目标网络在同步间隔内保持评估参数不变。Double DQN 中，在线网络选择的动作仍可能改变，因此目标值并非完全固定。两者都旨在改善神经网络学习的稳定性。测试时使用在线网络的贪心策略，不进行参数更新。

作业提供的默认配置使用一个约 **4.9 万个可训练参数**的 CNN；在线网络和目标网络各有一份相同结构的模型。默认在每张选定地图上采集 **30 万步**进行训练。

完成上述方法并安装 PyTorch 后，在 `PJ2-code-remake` 目录下训练 `smallClassic`：

```
python -m Agents.Runtime.dqn_agent --maps smallClassic
```

若已安装支持 CUDA 的 PyTorch 并确认 GPU 可用，可在命令末尾添加 `--device cuda`。

日志和模型保存在项目根目录的 `checkpoints` 下。测试时，将下面的路径替换为实际保存的模型文件；该命令会显示一局图形化游戏：

```
python -m Agents.Runtime.dqn_agent --load checkpoints/cnn_smallClassic_best_TIMESTAMP.pt --graphics --
    test-games 1 --map smallClassic
```

作为参考，使用作业提供的配置，模型应该能够学到通过获取宝剑斩杀恶龙，在 `smallClassic` 上能够达到平均分 1200 分以上、胜率约 80%。并且在未经训练的环境 `mediumClassic` 也上能够达到平均分约 1200 分、胜率约 80%。训练和测试存在随机性，具体结果可能有所不同。

手写题：Q8（2 分）MDP 的回报

MDP 过程的回报可写成以下形式：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{+\infty} \gamma^k R_{t+k+1}$$

设 MDP 过程无限进行，每一步的奖励均为固定的非零常数 r，且 $0 \leq \gamma < 1$。求 G_t，证明其与时间 t 无关，并说明它对 r 和 γ 的依赖关系。另讨论 $\gamma = 1$ 时该无穷级数是否收敛。